

# Ruffcol E-commerce Backend

## 1. Información General

- **Nombre del Proyecto:** Ruffcol E-commerce Backend
- **Descripción:** Sistema Web de Gestión y E-commerce para Tienda de Ropa y Accesorios de Mascotas.
- **Versión:** 1.0.0
- **Group ID:** `com.ruffcol`
- **Artifact ID:** `ruffcol-backend`
- **Repositorio de GitHub:** [RuffCol Pagina Y Recursos Adjuntos](#)

## 2. Stack Tecnológico

- **Lenguaje:** Java 17
- **Framework Principal:** Spring Boot 3.2.0
- **Persistencia:** Spring Data JPA
- **Seguridad:** Spring Security
- **Autenticación:** JWT 0.11.5 (JSON Web Tokens)
- **Base de Datos:** H2 (Entorno de desarrollo) / MySQL (Entorno de producción)
- **Documentación API:** SpringDoc OpenAPI 2.3.0 (Swagger)
- **Herramientas de Desarrollo:** Lombok (reducción de código boilerplate), Maven (gestión de dependencias).

## 3. Características Principales

- **Autenticación Segura:** Implementación de JSON Web Tokens (JWT).
- **Sistema de Roles:** Acceso basado en roles (`CLIENTE`, `ADMIN`, `SUPER_ADMIN`).
- **Gestión de Catálogo:** Administración de productos (soporte para subida de imágenes) y categorías.
- **Sistema de Pedidos:** Flujo de compra completo.
- **Estados de Pedido Controlados:** `PENDIENTE`, `EN_CONFECCION`, `ENVIADO`, `ENTREGADO`.
- **Control de Cuentas:** Estados de usuario (`PENDIENTE`, `ACTIVO`, `INACTIVO`).
- **Herramientas de Desarrollo:** Consola H2 integrada y documentación interactiva con Swagger/OpenAPI.
- **Configuración Web:** CORS habilitado para el frontend.

## 4. Configuración del Sistema

- Puerto del servidor: 8080
- JWT Secret:  
ruffcol-secret-key-2024-super-secreto-para-tokens-jwt
- JWT Expiración: 86400000ms (24 horas)
- Tamaño máximo de archivos (Uploads): 10MB
- Orígenes CORS permitidos: localhost:8080, localhost:3000
- URL Base de datos H2: jdbc:h2:mem:ruffcol\_db

## 5. Seguridad y Permisos

El sistema utiliza seguridad sin estado (stateless) mediante interceptación de peticiones HTTP.

- **Públicos (Sin Token):** Recursos estáticos (HTML/CSS/JS), endpoints de autenticación (Login/Registro), lectura de productos y categorías.
- **Rol CLIENTE:** Permisos para crear pedidos y visualizar el historial de sus propios pedidos.
- **Rol ADMIN:** Gestión completa de productos (CRUD y fotos), categorías y actualización de estados de pedidos.
- **Rol SUPER\_ADMIN:** Todos los privilegios del rol ADMIN, más la capacidad exclusiva de crear y gestionar cuentas de administradores.

## 6. Endpoints Principales (API REST)

### Autenticación (/api/auth)

- POST /register-cliente - Registro de nuevos clientes.
- POST /login - Inicio de sesión (Retorna el token JWT).

### Productos (/api/productos)

- GET / - Listar todos los productos.
- GET /{id} - Obtener detalles de un producto específico.
- GET /categoria/{idCategoria} - Filtrar productos por categoría.
- POST / - Crear un producto nuevo (Requiere ADMIN/SUPER\_ADMIN).
- POST / (multipart) - Crear producto y subir imagen adjunta.
- PUT /{id} - Actualizar datos de un producto (Requiere ADMIN/SUPER\_ADMIN).
- DELETE /{id} - Eliminar un producto (Requiere ADMIN/SUPER\_ADMIN).

### Categorías (/api/categorias)

- GET / - Listar todas las categorías.
- GET /{id} - Obtener detalles de una categoría.

- **POST /** - Crear una categoría *(Requiere ADMIN/SUPER\_ADMIN)*.
- **PUT /{id}** - Actualizar una categoría *(Requiere ADMIN/SUPER\_ADMIN)*.
- **DELETE /{id}** - Eliminar una categoría *(Requiere ADMIN/SUPER\_ADMIN)*.

## Pedidos (**/api/pedidos**)

- **POST /** - Crear un nuevo pedido *(Requiere Autenticación)*.
- **GET /mis-pedidos** - Listar los pedidos del usuario autenticado actual.
- **GET /** - Listar todos los pedidos globales *(Requiere ADMIN/SUPER\_ADMIN)*.
- **GET /{id}** - Obtener detalles de un pedido específico.
- **PUT /{id}/estado** - Actualizar el estado de un pedido *(Requiere ADMIN/SUPER\_ADMIN)*.
- **DELETE /{id}** - Eliminar un pedido *(Requiere ADMIN/SUPER\_ADMIN)*.

## Super Admin (**/api/superadmin**)

- **POST /crear-admin** - Crear directamente una cuenta de administrador *(Requiere SUPER\_ADMIN)*.
- **GET /administradores** - Listar todos los administradores activos.

## 7. Modelos de Datos (Entidades)

- **Usuario:** `id, nombre, email, password, rol, estadoCuenta, fechaRegistro`.
- **Producto:** `id, nombre, descripcion, precio, stock, imagenUrl, categoria`.
- **Categoria:** `id, nombre, descripcion, productos`.
- **Pedido:** `id, fechaPedido, estado, total, usuario, detalles`.
- **DetallePedido:** `id, cantidad, precioUnitario, producto, pedido`.

## 8. Enlaces y Recursos Locales

- **Servidor Base:** `http://localhost:8080`
- **Swagger UI (Interfaz Gráfica API):**  
`http://localhost:8080/swagger-ui.html`
- **API Docs (JSON):** `http://localhost:8080/api-docs`
- **Consola H2 (Base de Datos):** `http://localhost:8080/h2-console`

## 9. Instrucciones de Ejecución

Navegar al directorio del proyecto:

```
Bash
cd ruffcol-backend
```

### Opción A: Ejecutar en modo desarrollo con Maven:

Bash  
mvn spring-boot:run

### Opción B: Compilar y ejecutar el archivo JAR para producción:

Bash  
mvn clean package  
java -jar target/ruffcol-backend-1.0.0.jar

## 10. Estructura del Proyecto

Plaintext

ruffcol-backend/

```
|— src/main/java/com/ruffcol/tienda/
|   |— config/      # Configuraciones (Security, JWT, OpenAPI, CORS)
|   |— controlador/ # Controladores REST (Endpoints)
|   |— dto/         # Data Transfer Objects
|   |— exception/   # Manejo global de excepciones
|   |— modelo/      # Entidades JPA (Modelos de base de datos)
|   |— repositorio/ # Repositorios de Spring Data
|   |— servicio/    # Lógica de negocio
|   |— RuffcolApplication.java
|   |— src/main/resources/
|       |— static/      # Frontend Integrado (Archivos HTML, CSS, JS)
|       |— application.properties # Variables de entorno y configuración
|       |— schema-h2.sql # Esquema inicial de BD
|       |— data.sql     # Datos semilla (Seeders)
|   |— pom.xml         # Archivo de configuración de Maven
```